Artificial Intelligence
Lecture 4: Adversarial search

Prof. Dr. Pierre-Alexandre Murena

SoSe 2025

Outline

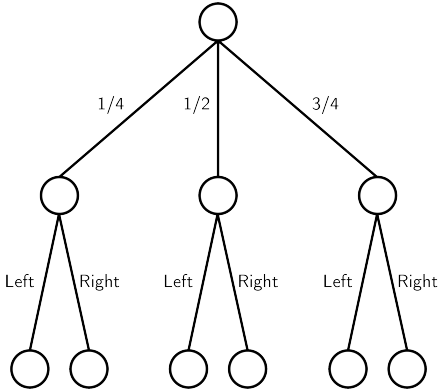
1. From single-agent to multi-agent
2. Solving adversarial problems
3. Two variants: Stochastic and partially observable games

Plan

1. From single-agent to multi-agent
   - A simple example
   - A short preliminary introduction to game theory
2. Solving adversarial problems
3. Two variants: Stochastic and partially observable games

Cutting cake for two

Alice wants to cut a cake to share it with Bob (unknown to her). Alice can cut the cake at 1/4, 1/2 or 3/4. Then, she decides which of the two slices (Left or Right) to give to Bob.



Goal: Get as much cake as possible

Plan: [1/4, Right]

Cutting cake for two

Variant: Now, Alice is still in charge of cutting the cake, but Bob is the one choosing his slice.

How does this affect Alice's cutting strategy?

1. **If Alice knows that Bob is hungry (and rude):**
   Alice knows that Bob will always select the biggest slice. It is better for her to cut at **1/2**.

Cutting cake for two

Variant: Now, Alice is still in charge of cutting the cake, but Bob is the one choosing his slice.

How does this affect Alice's cutting strategy?

1. **If Alice knows that Bob is hungry (and rude):**
   Alice knows that Bob will always select the biggest slice. It is better for her to cut at **1/2**.
2. **If Alice knows that Bob will be polite:**
   Alice knows that Bob will always select the smallest slice. It is better for her to cut at **1/4** or **3/4**. *(Unless she wants to be polite)*

Cutting cake for two

Variant: Now, Alice is still in charge of cutting the cake, but Bob is the one choosing his slice.

How does this affect Alice's cutting strategy?

1. **If Alice knows that Bob is hungry (and rude):**
   Alice knows that Bob will always select the biggest slice. It is better for her to cut at **1/2**.
2. **If Alice knows that Bob will be polite:**
   Alice knows that Bob will always select the smallest slice. It is better for her to cut at **1/4** or **3/4**. *(Unless she wants to be polite)*
3. **If Alice doesn't know:** ?

Cutting cake for two

If Alice cannot predict Bob's behaviour:

The safest option is to think of the worst case:

- If she plays 1/4 or 3/4, in the worst case Bob will choose the biggest slice, and she will get only 1/4.
- If she plays 1/2, Bob will choose any of the slices, and Alice will get 1/2.

Adversarial setting

The cake cutting task is called **adversarial**:
The two agents have opposed interests: *The more cake Alice gets, the less cake Bob gets.*

Many games you can think of are adversarial: chess, go, checkers, tic-tac-toe...

Adversarial setting

The cake cutting task is called **adversarial**:
The two agents have opposed interests: *The more cake Alice gets, the less cake Bob gets.*

Many games you can think of are adversarial: chess, go, checkers, tic-tac-toe...

The cake cutting task is formalized as a **game**:
Unlike a search task as those encountered until now, the success is not dependent on the agent's actions only, but also on the other agents' actions.

Plan

1. From single-agent to multi-agent
   - A simple example
   - A short preliminary introduction to game theory
2. Solving adversarial problems
3. Two variants: Stochastic and partially observable games

What is a game?

A **game** is defined as a multi-agent decision problem defined as follows:

Players: The participating agents

What is a game?

A **game** is defined as a multi-agent decision problem defined as follows:

Players: The participating agents

A player function: Determining which player takes action at each state

What is a game?

A **game** is defined as a multi-agent decision problem defined as follows:

Players: The participating agents

A player function: Determining which player takes action at each state

An action function: Returns the set of actions available to the current player in a given state

What is a game?

A **game** is defined as a multi-agent decision problem defined as follows:

Players: The participating agents

A player function: Determining which player takes action at each state

An action function: Returns the set of actions available to the current player in a given state

A transition model: Describes the effect of an action over the state

What is a game?

A **game** is defined as a multi-agent decision problem defined as follows:

Players: The participating agents

A player function: Determining which player takes action at each state

An action function: Returns the set of actions available to the current player in a given state

A transition model: Describes the effect of an action over the state

Terminal test: Checks when the game ends. States that pass the test are called **terminal states**.

What is a game?

A **game** is defined as a multi-agent decision problem defined as follows:

Players: The participating agents

A player function: Determining which player takes action at each state

An action function: Returns the set of actions available to the current player in a given state

A transition model: Describes the effect of an action over the state

Terminal test: Checks when the game ends. States that pass the test are called **terminal states**.

Utility function: Defines the payoff of each player in a terminal state.

Example: Formalizing the cake cutting example

Players: $\mathcal{P} = \{\text{Alice}, \text{Bob}\}$

Example: Formalizing the cake cutting example

Players: $\mathcal{P} = \{\text{Alice}, \text{Bob}\}$

Player function: Alice taking action first, then Bob

Example: Formalizing the cake cutting example

Players: $\mathcal{P} = \{\text{Alice}, \text{Bob}\}$

Player function: Alice taking action first, then Bob

Action function:

- $\textsc{Actions}(\text{Alice}) = \{1/4, 1/2, 3/4\}$
- $\textsc{Actions}(\text{Bob}) = \{L, R\}$

Example: Formalizing the cake cutting example

Players: $\mathcal{P} = \{\text{Alice}, \text{Bob}\}$

Player function: Alice taking action first, then Bob

Action function:

- $\textsc{Actions}(\text{Alice}) = \{1/4, 1/2, 3/4\}$
- $\textsc{Actions}(\text{Bob}) = \{L, R\}$

Transition model: Cake being cut

Example: Formalizing the cake cutting example

Players: $\mathcal{P} = \{\text{Alice}, \text{Bob}\}$

Player function: Alice taking action first, then Bob

Action function:

- $\textsc{Actions}(\text{Alice}) = \{1/4, 1/2, 3/4\}$
- $\textsc{Actions}(\text{Bob}) = \{L, R\}$

Transition model: Cake being cut

Terminal test: After Bob's action

Example: Formalizing the cake cutting example

Players: $\mathcal{P} = \{\text{Alice}, \text{Bob}\}$

Player function: Alice taking action first, then Bob

Action function:

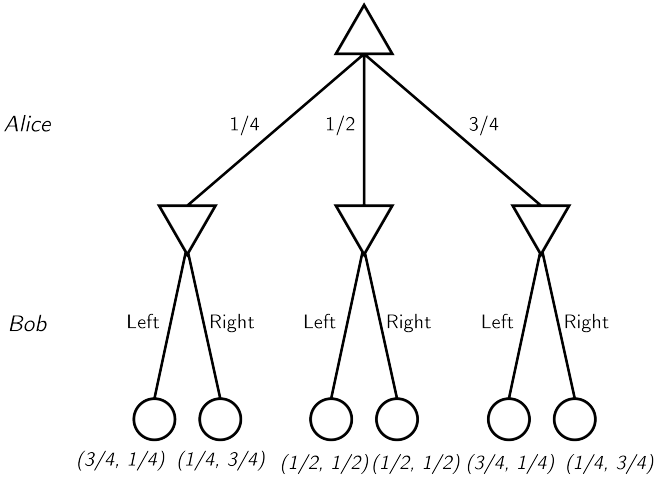
- $\textsc{Actions}(\text{Alice}) = \{1/4, 1/2, 3/4\}$
- $\textsc{Actions}(\text{Bob}) = \{L, R\}$

Transition model: Cake being cut

Terminal test: After Bob's action

Utility function: $v(p) =$ quantity of cake that player p gets

Example: Formalizing the cake cutting example



Zero-sum games

Many games have a winner and a loser. In other words, in such games, the gain of the winner is the loss of the loser.

We call such games **zero-sum games**, and we define them formally as games such that in any terminal state s:

$$v(s, P_1) + v(s, P_2) = 0$$

Zero-sum games

Many games have a winner and a loser. In other words, in such games, the gain of the winner is the loss of the loser.

We call such games **zero-sum games**, and we define them formally as games such that in any terminal state s:

$$v(s, P_1) + v(s, P_2) = 0$$

Remarks:

- Zero-sum games are **adversarial**: the opponent aims to minimize the gains of the player
- With this condition, it is sufficient to specify the value for one player.

Outline

1 From single-agent to multi-agent

2 Solving adversarial problems

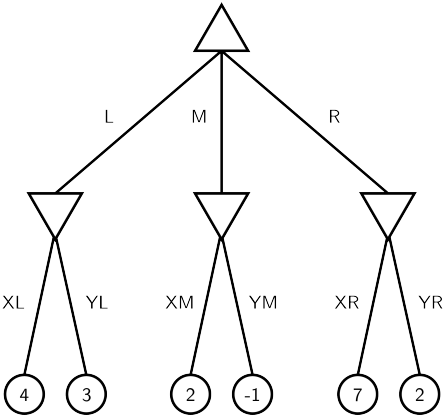
3 Two variants: Stochastic and partially observable games

Plan

1. From single-agent to multi-agent
2. Solving adversarial problems
   - The minimax algorithm
   - Backward induction
   - Alpha-beta pruning
   - Imperfect real-time decisions
3. Two variants: Stochastic and partially observable games

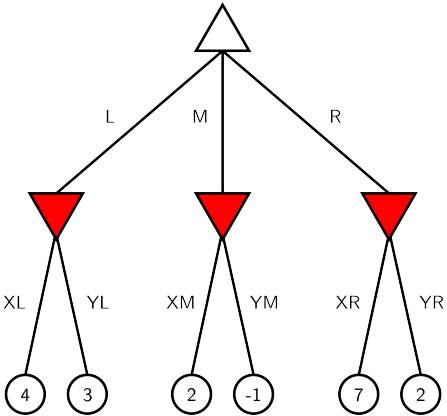
An example

Question: Which action to choose to maximize the value?



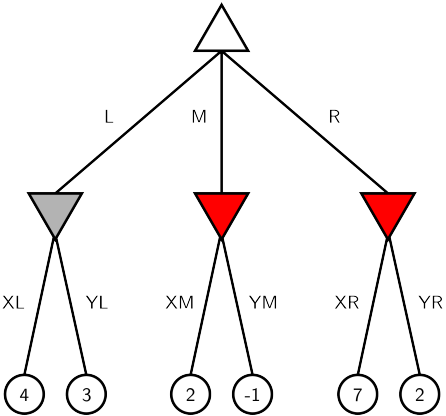
An example

Step 2: What happens if Player 1 plays Left?



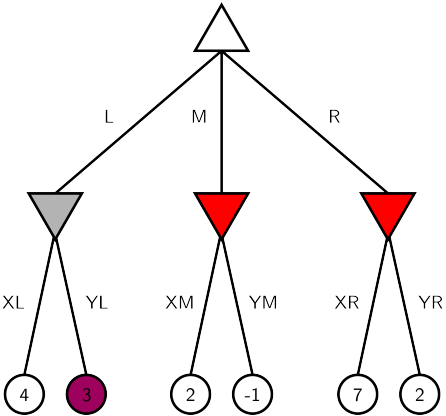
An example

Step 3: What would Player 2 play?



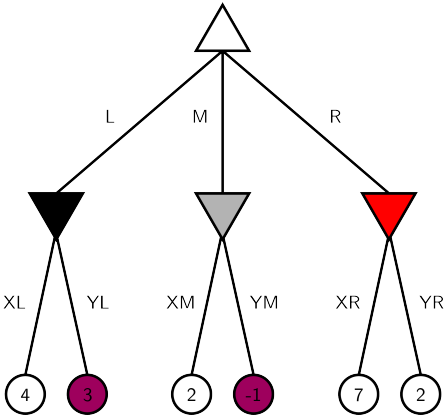
An example

Step 4: Player 2 will aim to minimize my gain, therefore will play YL.



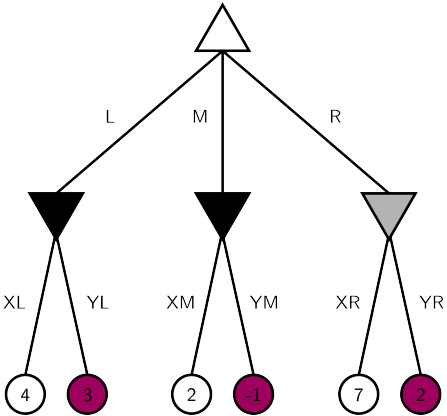
An example

Step 5: If Player 1 plays Middle, Player 2 should play YM



An example

Step 6: If Player 1 plays Right, Player 2 should play YR



Minimax algorithm

To choose which action to play, we need to know the value of the resulting states. The value can be computed as follows:

- **If s is a terminal state:** The value is $v(s)$.

Minimax algorithm

To choose which action to play, we need to know the value of the resulting states. The value can be computed as follows:

- **If s is a terminal state:** The value is $v(s)$.
- **If the agent is playing at s:** The value is the maximal value of the successor states:

$$v(s) = \max_{s' \in \text{Successors}(s)} v(s')$$

Minimax algorithm

To choose which action to play, we need to know the value of the resulting states. The value can be computed as follows:

- **If s is a terminal state:** The value is $v(s)$.
- **If the agent is playing at s:** The value is the maximal value of the successor states:

$$v(s) = \max_{s' \in \text{Successors}(s)} v(s')$$

- **If the opponent is playing at s:** The value is the minimal value of the successor states:

$$v(s) = \min_{s' \in \text{Successors}(s)} v(s')$$

Minimax implementation

```
def value(state):
    if state is terminal: return utility
    if next agent is MAX (playing agent): return max_value(state)
    if next agent is MIN (opponent): return min_value(state)
```

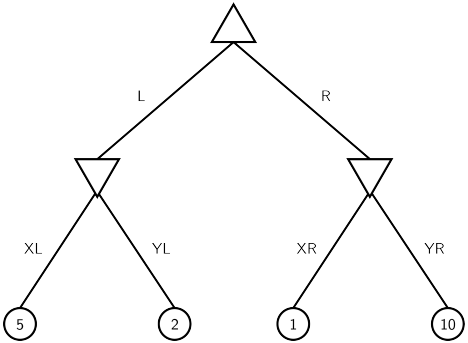
```
def max_value(state):
    initialize v = - ∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min_value(state):
    initialize v = ∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

Optimality of minimax

By construction, minimax is optimal if the opponent is playing rationally (i.e. using minimax).

What if the opponent is not perfect?



An interesting property

If the opponent is still playing rationally (i.e. using minimax), the agent has no interest in changing strategy. The same is true for the opponent. This property has a name in game theory: this is a **Nash equilibrium**.

An interesting property

If the opponent is still playing rationally (i.e. using minimax), the agent has no interest in changing strategy. The same is true for the opponent. This property has a name in game theory: this is a **Nash equilibrium**.

An example of a Nash equilibrium: Two friends want to watch a movie, but disagree on which movie. But still, they prefer watching any movie together than their favorite movie alone.

The choice of going to watch either movie together is a Nash equilibrium.

Plan

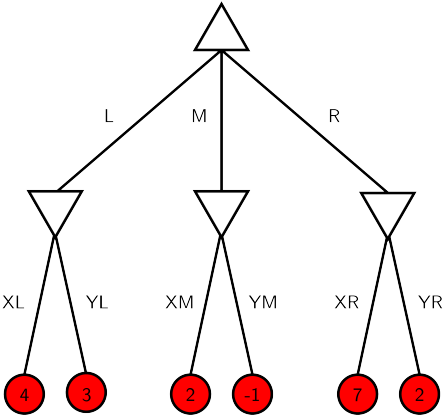
1. From single-agent to multi-agent
2. Solving adversarial problems
   - The minimax algorithm
   - Backward induction
   - Alpha-beta pruning
   - Imperfect real-time decisions
3. Two variants: Stochastic and partially observable games

Backward induction

In the minimax algorithm, we explore the tree from the root. In **backward induction**, we explore the tree from the terminal nodes and backpropagate the value in the tree.

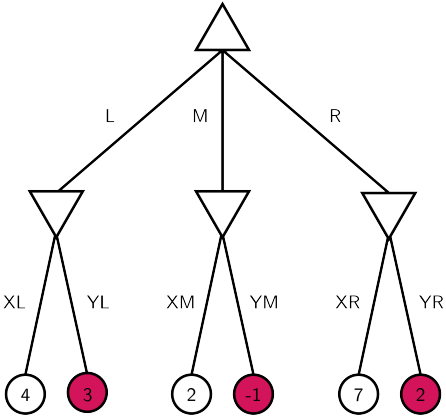
Example

Step 1: We select the terminal nodes.



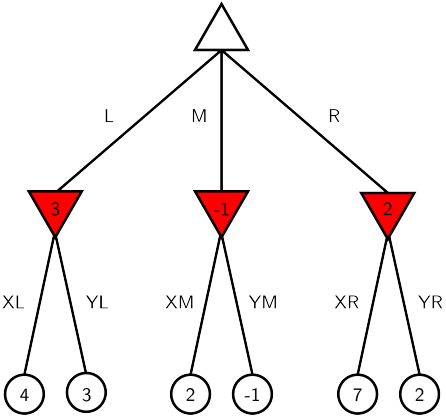
Example

Step 2: We select the optimal values for the opponent.



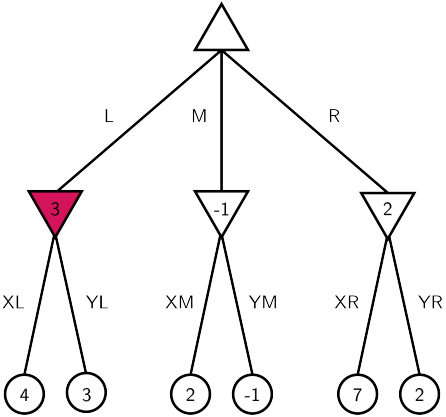
Example

Step 3: We backpropagate the values to the parent nodes.



Example

Step 4: We select the optimal values for the agent.



Minimax vs Backward propagation

Minimax is a **top-down** technique: it explores (recursively) from the root to the terminal nodes.

Backward induction is a **bottom-up** technique: it explores from the terminal nodes to the root (using backpropagation).

Minimax vs Backward propagation

Minimax is a **top-down** technique: it explores (recursively) from the root to the terminal nodes.

Backward induction is a **bottom-up** technique: it explores from the terminal nodes to the root (using backpropagation).

However, both algorithms are doing the same: backpropagating the terminal utilities as values of the nodes in the tree, assuming that both agents play optimally.

Plan

1. From single-agent to multi-agent
2. Solving adversarial problems
   - The minimax algorithm
   - Backward induction
   - Alpha-beta pruning
   - Imperfect real-time decisions
3. Two variants: Stochastic and partially observable games

The main problem with minimax

The main problem with minimax is that it requires to explore all nodes, which can be very long for large state spaces.

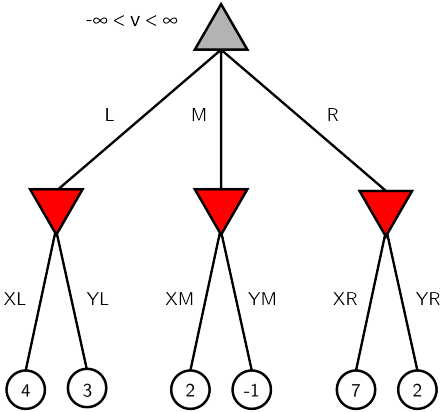
Examples: Number of nodes in the tree graph for...

- **Tic-tac-toe:** $9! = 362880$
- **Chess:** $\sim 10^{40}$
- **Go:** $\sim 10^{170}$ for a 19×19 board

It would be amazing if we could avoid exploring all nodes... Let's re-run minimax and see if we couldn't eliminate some branches...

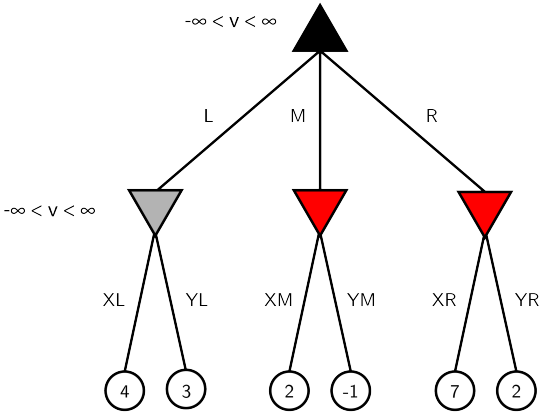
Going through minimax again...

Step 1: We want to know the value of the root node, but have no information about it



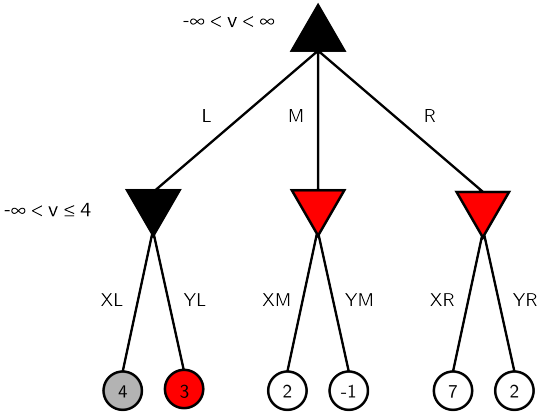
Going through minimax again...

Step 2: Therefore, we compute the value of the first child node



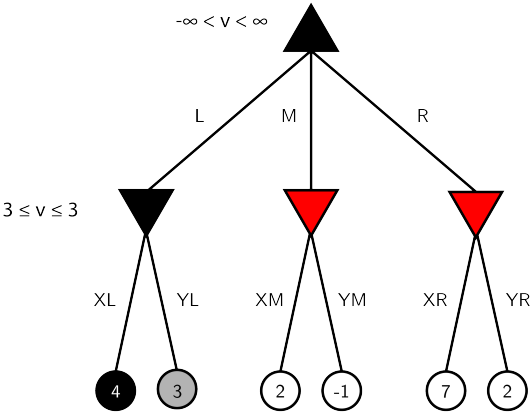
Going through minimax again...

Step 3: We are sure that this value is lower than 4



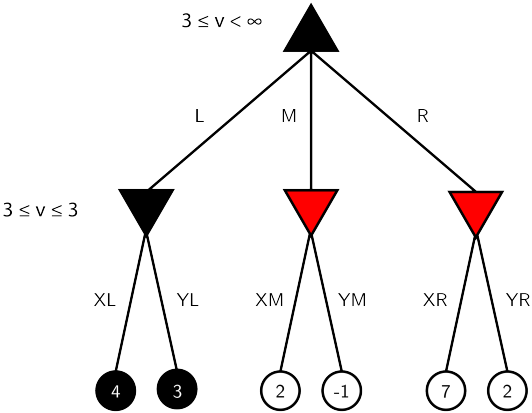
Going through minimax again...

Step 4: It is actually equal to 3 (since all children have been explored)



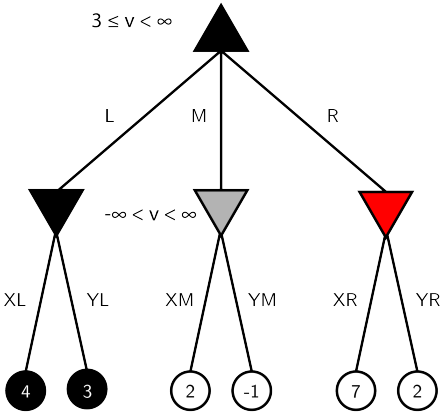
Going through minimax again...

Step 4: We backpropagate the information to the root node (MAX node)



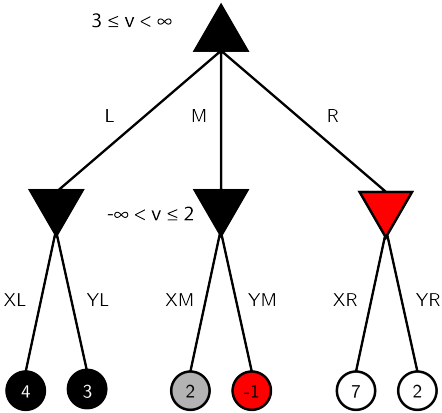
Going through minimax again...

Step 5: We now compute the value of the second child node



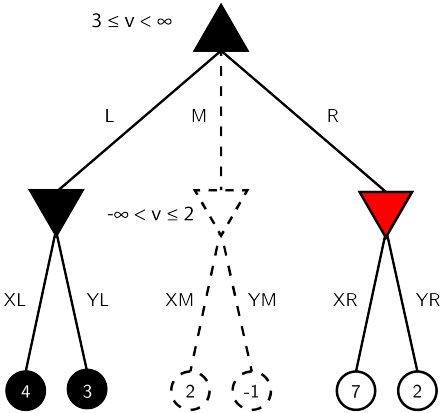
Going through minimax again...

Step 6: We are sure that this value is lower than 2



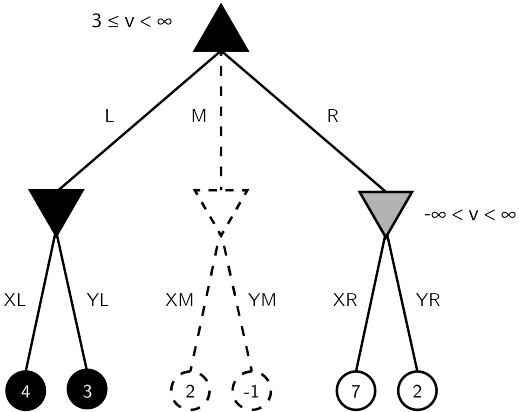
Going through minimax again...

Step 7: No need to explore more: the value of this node is too low! We prune branch M.



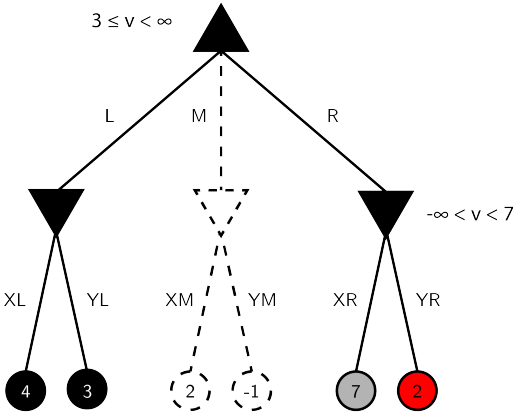
Going through minimax again...

Step 8: We now compute the value of the third (and last) child node



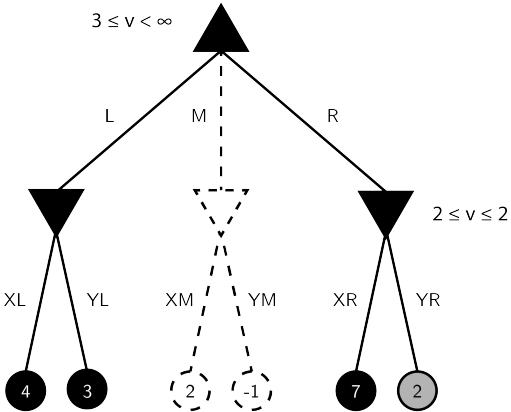
Going through minimax again...

Step 9: We are sure that this value is lower than 7. This does not allow us to prune.



Going through minimax again...

Step 10: It is actually equal to 3 (since all children have been explored)



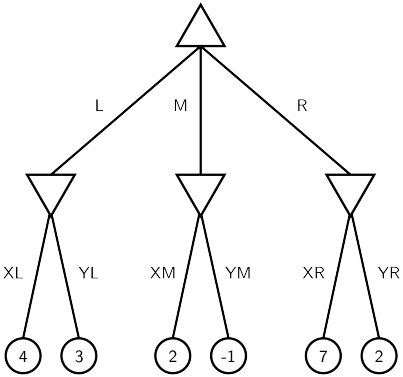
Alpha-beta pruning

Alpha-beta pruning is a variant of the minimax algorithm which keeps tracks of lower and upper bounds of the value along a path, and prunes branches that do not fit into these bounds.

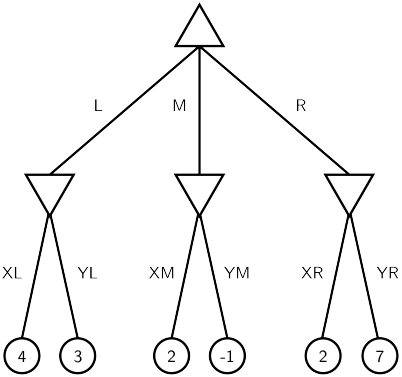
```
def max_value(state, α, ß):
    initialize v = - ∞
    for each successor of state:
        v = max(v, value(successor, α, ß))
        if v ≥ ß: return v
        α = max(α, v)
    return v
```

```
def min_value(state):
    initialize v = ∞
    for each successor of state:
        v = min(v, value(successor, α, ß))
        if v ≤ α: return v
        ß = min(ß, v)
    return v
```

The exploration order matters!



5 terminal nodes explored



4 terminal nodes explored

Underlying hypotheses

The minimax algorithm is designed for a very specific type of problem with the following restrictions:

- **Sequential decision making:** The players play one after the other
- **Full observability:** The players know everything about what happened, and know the transition model.
- **The adversary:**
  - knows the environment
  - knows that the agent plays rationally
  - plays rationally

Plan

1. From single-agent to multi-agent
2. Solving adversarial problems
   - The minimax algorithm
   - Backward induction
   - Alpha-beta pruning
   - Imperfect real-time decisions
3. Two variants: Stochastic and partially observable games

An analogy: uninformed vs informed search

In **uninformed search**, the agent explores the tree without having any information over the potential success of a branch.

In **informed search**, the agent has access to some predictions over the potential success of their actions, before testing them.

An analogy: uninformed vs informed search

In **uninformed search**, the agent explores the tree without having any information over the potential success of a branch.

In **informed search**, the agent has access to some predictions over the potential success of their actions, before testing them.

This is similar in games:

The minimax algorithm explores the whole game tree to determine which action to play without any information about their potential outcomes.

Instead, we can use a **heuristic evaluation function** that estimates the value of non-terminal nodes.

Evaluation functions

Ideal case: The evaluation function provides the actual value of the state.

In practice, we typically use a linear combination of **features**:

$$\textsc{Eval}(s) = \sum_i w_i f_i(s)$$

where f_i are features of the state and w_i are well-chosen weights

Outline

1 From single-agent to multi-agent

2 Solving adversarial problems

3 Two variants: Stochastic and partially observable games

Plan

1. From single-agent to multi-agent
2. Solving adversarial problems
3. Two variants: Stochastic and partially observable games
   - Stochastic games
   - Partially Observable Games

A variant of the cake cutting game

We consider the following variant of the cutting cake game, where Alice and Bob both aim to get as much cake as possible.

1. Alice cuts the cake at 1/4, 1/2 or 3/4.
2. Alice and Bob throw a dice and, depending on the result:
   - Alice chooses her slice; or
   - Bob chooses his slice.

A variant of the cake cutting game

We consider the following variant of the cutting cake game, where Alice and Bob both aim to get as much cake as possible.

1. Alice cuts the cake at 1/4, 1/2 or 3/4.
2. Alice and Bob throw a dice and, depending on the result:
   - Alice chooses her slice; or
   - Bob chooses his slice.

Remark: Since Alice doesn't try to reach a specific final goal but to maximize her utility when reaching a terminal state, the AND-OR search is not adapted.

Dependence to chance

Alice's choice depends on the condition on the dice for her to choose her slice.

Condition 1: Get a 6

In this case, it is very unlikely that she can choose her slice. She cuts the cake at $1/2$.

Dependence to chance

Alice's choice depends on the condition on the dice for her to choose her slice.

Condition 1: Get a 6
In this case, it is very unlikely that she can choose her slice. She cuts the cake at 1/2.

Condition 2: Get 1, 2, 3, 4 or 5
In this case, it is very likely that she can choose her slice. She cuts the cake at 1/4 or 3/4.

Dependence to chance

Alice's choice depends on the condition on the dice for her to choose her slice.

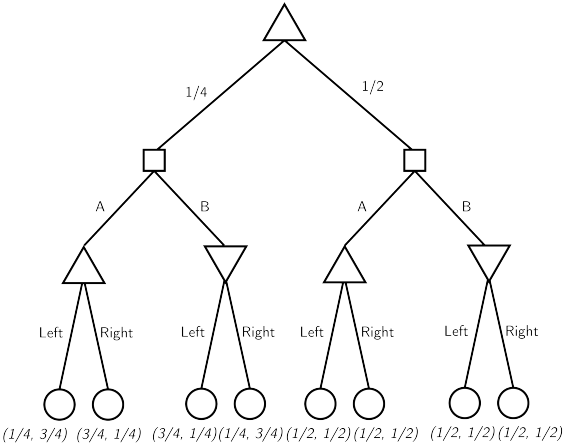
Condition 1: Get a 6
In this case, it is very unlikely that she can choose her slice. She cuts the cake at 1/2.

Condition 2: Get 1, 2, 3, 4 or 5
In this case, it is very likely that she can choose her slice. She cuts the cake at 1/4 or 3/4.

Condition 3: Get 4, 5 or 6
She has equal chances to choose and not to choose.

The Chance player

We model the chance by a third player, called **Chance**.



Modeling chance

Which strategy does the chance player follow?

Modeling chance

Which strategy does the chance player follow?

The chance player is **not** adversarial: the chance doesn't try to prevent Alice from winning.

Representing the chance

It might be interesting for the agent to know what will be the outcome of chance.

This is done with **probabilities**.

For instance, the probability of Alice to get selected by chance would be $1/6$ if she can decide when drawing a 6.

The value of this chance node is then computed as follows:

$$v = p(\text{Alice is selected}) \times \frac{3}{4} + p(\text{Alice is not selected}) \times \frac{1}{2}$$

This is called the **Expectiminimax algorithm**.

Expectiminimax algorithm

We generalize the notion of **value** of a deterministic game to the notion of **expectiminimax** for stochastic games:

$$\textsc{Expectiminimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } s \text{ is a terminal state} \\ \max_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{the player is the agent } (\textsc{Max}) \\ \min_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{if the player is the adversary } (\textsc{Min}) \end{cases}$$

Expectiminimax algorithm

We generalize the notion of **value** of a deterministic game to the notion of **expectiminimax** for stochastic games:

$$\textsc{Expectiminimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } s \text{ is a terminal state} \\ \max_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{the player is the agent } (\textsc{Max}) \\ \min_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{if the player is the adversary } (\textsc{Min}) \\ \sum_r P(r)\textsc{Expectiminimax}(\textsc{Result}(s,r)) & \text{the player is } \textsc{Chance} \end{cases}$$

Plan

1. From single-agent to multi-agent
2. Solving adversarial problems
3. Two variants: Stochastic and partially observable games
   - Stochastic games
   - Partially Observable Games

Deterministic partially observable games

In **deterministic partially observable games**, uncertainty about the state of the game is a consequence of the lack of observability of the adversary's actions.

Example: Kriegspiel

Kriegspiel is a variant of chess where the players can only see:

- the position of their own pieces;
- if a proposed move is legal or not;
- the outcome of a legal move (nothing, captures a piece, check)

Belief states for deterministc PO games

Belief state: set of all possible states the game can be in

State estimation: Based on the current belief state and the observation of the outcomes, the player updates their estimation of the belief state.

Guaranteed strategy: The player chooses an action that is **guaranteed** to lead to victory, in whichever state we can be in the current belief state.

Probabilistic strategy: The player chooses an action following a probability distribution over actions. Such a distribution is called a **mixed strategy**.

Stochastic partial observability

In **stochastic partially observable games**, the missing information is generated randomly.

Example: In card games, the partial observability is due to the unobserved ordering of the cards.

A solution: The player chooses an action that is optimal *on average* over the possible states:

$$\arg\max_a \sum_s P(s)\textsc{Minimax}(\textsc{Result}(s,a))$$

To know more...

Adversarial search:

- Norvig and Russell, chapter 5

Alpha-beta pruning:

- Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning.

A standard example: chess:

- Campbell, M., et al. (2002). Deep Blue.
- Silver, D., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm.
- Tomasev, N., et al. (2022). Reimagining chess with AlphaZero.